

Week 38: Automatic Differentiation, Adaptive Learning Rates and Stochastic Gradient Descent

Gradients without deriving them; AdaGrad, RMSProp and Adam; and why a random subset of the data is enough

With pauses for questions

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for week 38

Monday lecture (2×45 min):

- ▶ **First 10 min:** gradient descent with and without momentum, once more – following the steps on a low-order polynomial (Sections 4.3, 4.5 and 4.6; `week38.ipynb`)
- ▶ Automatic differentiation: the gradient without deriving it, forward and reverse mode, JAX on last week's problems (Section 4.14 – moved here from week 37, where we ran out of time)
- ▶ Why one learning rate cannot fit all directions, and the three adaptive methods: AdaGrad, RMSProp and Adam (Sections 4.8–4.12)
- ▶ Stochastic gradient descent: minibatches, epochs, what the noise means, learning-rate schedules and stopping (Section 4.7)

Tuesday session (2×45 min, flipped): your questions, then two cases – AdaGrad, RMSProp and Adam in your own code, on the Runge problem of Project 1; and stochastic gradient descent with minibatches, epochs and schedules. Bring your laptop.

Reading

Chapter 4, Sections 4.7–4.14 of the book. Goodfellow et al., Sections 8.1–8.5; Kingma and Ba, *Adam: a method for stochastic optimization* (2014). Week 39: Lasso by gradient methods and logistic regression (Chapter 5).

Where we were: one step, one number

Gradient descent, Eq. (4.10):

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma \nabla C(\boldsymbol{\theta}_k).$$

For a quadratic cost with Hessian H the error decouples along the eigendirections, Eq. (4.19): component i contracts by $|1 - \gamma \lambda_i|$ per step. Hence (Theorem 4.1)

$$\text{stable} \iff 0 < \gamma < \frac{2}{\lambda_{\max}} \quad \text{Eq. (4.20),}$$

and the slow direction $\lambda_{\min}$ sets the iteration count, $\propto \kappa = \lambda_{\max}/\lambda_{\min}$, Eq. (4.26). For regression $\kappa(H) = \kappa_2(X)^2$, Eq. (4.27).

Momentum, Eq. (4.28):

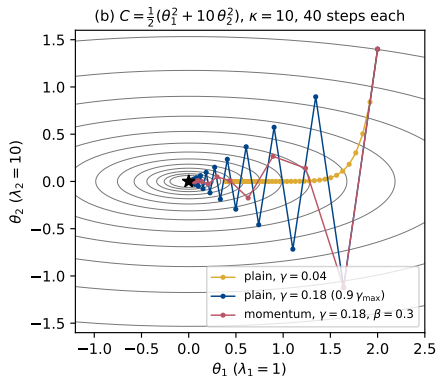
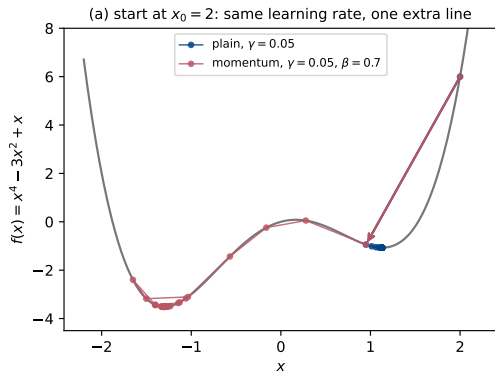
$$\mathbf{v}_{k+1} = \beta \mathbf{v}_k + \gamma \nabla C(\boldsymbol{\theta}_k), \quad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mathbf{v}_{k+1}.$$

An exponentially weighted sum of all gradients, Eq. (4.29): consistent gradients add up (effective step $\gamma/(1 - \beta)$, Eq. 4.30), alternating ones cancel. Rate $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ with tuned γ, β , Eq. (4.31).

The two numbers to remember

$\gamma < 2/\lambda_{\max}$ decides whether it converges; κ decides how long it takes. Everything today is an attempt to escape the second without violating the first.

Following the steps on a polynomial (week38.ipynb)



(a)

$f(x) = x^4 - 3x^2 + x$, minima at $x = -1.30$ (global, $f = -3.51$) and $x = 1.13$ (local, $f = -1.07$), start $x_0 = 2$, $\gamma = 0.05$. Plain descent walks down into the *nearest* minimum and stays:

$2 \rightarrow 0.95 \rightarrow 1.01 \rightarrow 1.06 \rightarrow \dots \rightarrow 1.131$. With $\beta = 0.7$ the velocity carries the iterate over the hump at $x = 0.17$: $2 \rightarrow 0.95 \rightarrow 0.28 \rightarrow -0.16 \rightarrow -0.57 \rightarrow \dots \rightarrow -1.302$. Same code, one extra line, a different minimum.

(b) $C = \frac{1}{2}(\theta_1^2 + 10\theta_2^2)$, $\kappa = 10$, $\gamma_{\max} = 0.2$. Iterations to 10^{-6} : $\gamma = 0.04$: 356; $\gamma = 0.18$: 74, zigzagging across the steep direction; momentum $\beta = 0.3$ at the same γ : 39 (tuned $\gamma = 0.231$, $\beta = 0.27$: 28, against the $\sqrt{10}$ ceiling).

The code that follows the steps

```
def quartic(x): return x**4 - 3.0 * x**2 + x
def quartic_grad(x): return 4.0 * x**3 - 6.0 * x + 1.0

def descend(grad, x0, gamma, beta=0.0, num_iters=100):
    """Gradient descent with momentum, Eq. (4.28); beta = 0 is plain gradient descent, Eq. (4.10).
    Works for a scalar x as well as for a vector theta. Returns every iterate."""
    x, v = np.array(x0, dtype=float), np.zeros_like(np.array(x0, dtype=float))
    history = [x.copy()]
    for _ in range(num_iters):
        v = beta * v + gamma * grad(x)
        x = x - v
        history.append(x.copy())
    return np.array(history)

path = descend(quartic_grad, 2.0, gamma=0.05, beta=0.7, num_iters=40)
for k, x in enumerate(path[:8]):
    print(f"step {k}: x = {x:8.4f} f(x) = {quartic(x):8.4f} f'(x) = {quartic_grad(x):8.4f}")
```

Print the table, then change γ , β and x_0 and watch: this is the whole of the notebook's first section, and the mental model for everything that follows. Note the *gradient is an argument*: today's first topic is where it comes from when nobody derived it.

Pause and think 1: the hump

Question

In panel (a) momentum found the global minimum and plain gradient descent did not. Is momentum therefore a method for finding global minima? What would happen with $\beta = 0.9$?

Pause and think 1: the hump

Question

In panel (a) momentum found the global minimum and plain gradient descent did not. Is momentum therefore a method for finding global minima? What would happen with $\beta = 0.9$?

Answer

No. Momentum has no knowledge of the landscape beyond the gradients it has seen; it rolled over the hump because the velocity built up on the steep slope from $x_0 = 2$ happened to exceed the barrier. Start at $x_0 = 0.5$ and it falls into whichever basin is nearer; with $\beta = 0.9$ the ball is so frictionless that it overshoots the global minimum, climbs back over the hump and ends up in the *local* basin (the notebook shows the path: $\dots, -2.12, -1.29, -0.55, \dots$). For a non-convex cost none of this week's methods comes with a guarantee – Section 4.1's “stationary point = global minimum” is the property of convexity, and we left it behind at the first quartic. What noise does for this problem: Part 4.

Every method this week needs a gradient (Section 4.14)

For OLS we derived it by hand, Eq. (4.13), and last week every method used that formula. For a network with a dozen layers that is possible but error-prone; for a model under active development, impractical. **Automatic differentiation** (AD) solves the problem completely.

The idea

Every program, however complicated, executes a sequence of elementary operations – $+$, $\times$, $\exp$, $\log$, $\sin$ – each with a known derivative. Apply the chain rule to that sequence and you obtain derivatives of any order, **accurate to working precision**, at a cost only a small constant factor above the program itself. Both halves of that sentence are theorems.

Plan for the next twenty-five minutes: what AD is *not* (symbolic, numerical); the evaluation trace; the two modes, forward and reverse, from the chain rule; the cheap gradient principle; JAX on last week's problems. *Reverse-mode AD is what backpropagation is*. Chapter 8 will meet it again under that name.

Three ways to differentiate a program (Section 4.14.1)

Symbolic. Manipulates the *formula*; exact, but needs a single closed-form expression (no loops, no branches) and suffers expression swell. **Numerical.** Central difference, Eq. (4.57),

$$D_h f(x) = \frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6} f'''(x) + \dots$$

Truncation error $\propto h^2$ (Eq. 4.63) wants h small; rounding error $\epsilon_M |f|/h$ wants it large:

$$E(h) \approx \frac{h^2}{6} |f'''| + \frac{\epsilon_M |f|}{h}, \quad \text{Eq. (4.59).}$$

Minimising, Eq. (4.60):

$$h^* \sim \epsilon_M^{1/3} \approx 6 \times 10^{-6}, \quad E(h^*) \sim \epsilon_M^{2/3} \approx 4 \times 10^{-11}.$$

Two thirds of the digits at best, and only if h is chosen well (one-sided: $\epsilon_M^{1/2} \approx 10^{-8}$). And the cost: $2p$ function evaluations per gradient. For $p = 10^7$, a week of computing for one mediocre gradient.

Automatic differentiation has neither defect

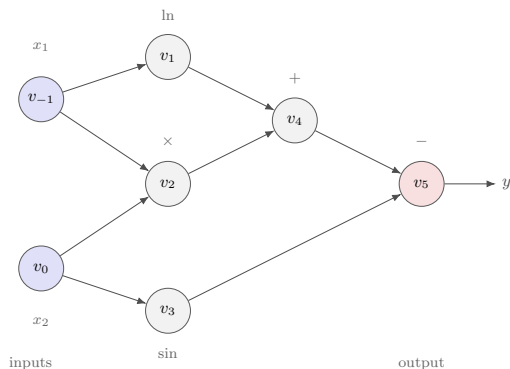
It computes the *value* of the derivative at a point (no swell), by propagating exact rules through the program (no h , no truncation), and in reverse mode obtains all p components for a small constant number of function evaluations.

The evaluation trace and the computational graph (Section 4.14.2)

AD works not on the formula for f but on the operations a program executes. For $f(x_1, x_2) = \ln x_1 + x_1 x_2 - \sin x_2$, Eq. (4.61), the **evaluation trace** (Wengert list), Eq. (4.62):

$$\begin{aligned}v_{-1} &= x_1, & v_0 &= x_2, \\v_1 &= \ln v_{-1}, & v_2 &= v_{-1} v_0, & v_3 &= \sin v_0, \\v_4 &= v_1 + v_2, & v_5 &= v_4 - v_3, & y &= v_5.\end{aligned}$$

Drawn with an edge $j \rightarrow i$ whenever v_i uses v_j : the **computational graph** (book Figure 4.8). Loops unroll, branches select – *any* program has a trace. A neural network is a particularly regular one. Each node's **local derivatives** are known in closed form, Eq. (4.63): $\partial v_1 / \partial v_{-1} = 1/v_{-1}$, $\partial v_2 / \partial v_{-1} = v_0$, $\partial v_2 / \partial v_0 = v_{-1}$, $\partial v_3 / \partial v_0 = \cos v_0$, the rest ± 1 . **All AD does is combine these by the chain rule. The only question is in which order.**



Forward mode: tangents (Section 4.14.3)

Fix a direction $\dot{x}$ in input space and carry with every node its **tangent**, the directional derivative

$$\dot{v}_i = \sum_k \frac{\partial v_i}{\partial x_k} \dot{x}_k, \quad \text{Eq. (4.64),} \quad \text{by the chain rule} \quad \dot{v}_i = \sum_{j \prec i} \frac{\partial \varphi_i}{\partial v_j} \dot{v}_j, \quad \text{Eq. (4.65).}$$

Runs in the *same order* as the evaluation, so each tangent is computed alongside its value.
With $\dot{x} = (1, 0)$ at $(x_1, x_2) = (2, 5)$, Eq. (4.66):

$$\begin{aligned} \dot{v}_{-1} &= 1, & \dot{v}_0 &= 0, \\ \dot{v}_1 &= \dot{v}_{-1}/v_{-1} = 0.5, & \dot{v}_2 &= \dot{v}_{-1}v_0 + v_{-1}\dot{v}_0 = 5, & \dot{v}_3 &= \cos(v_0)\dot{v}_0 = 0, \\ \dot{v}_4 &= 5.5, & \dot{v}_5 &= 5.5 = \partial f / \partial x_1. \end{aligned}$$

For $\partial f / \partial x_2$: repeat the whole sweep with $\dot{x} = (0, 1)$.

Cost

One sweep returns $\dot{y} = J\dot{x}$, a *Jacobian–vector product*, Eq. (4.69), without forming J . The gradient of a scalar function of p parameters costs p sweeps – the count of finite differences, only exact. Right when the *inputs* are few; wrong for training.

Dual numbers: forward mode in one line (Section 4.14.3)

Introduce ε with $\varepsilon^2 = 0$, $\varepsilon \neq 0$, and the **dual numbers** $a + b\varepsilon$:

$$(a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon, \quad \text{Eq. (4.67),}$$

the product rule as an algebra. For any g with a Taylor series,

$$g(a + b\varepsilon) = g(a) + g'(a)b\varepsilon + \frac{1}{2}g''(a)b^2\varepsilon^2 + \cdots = g(a) + g'(a)b\varepsilon, \quad \text{Eq. (4.68).}$$

The series *terminates* – by the algebra of ε , not by neglecting anything. Evaluate g at $x + 1 \cdot \varepsilon$ and read off the coefficient of ε : $g'(x)$, exactly.

What this buys

A dual number carries a value and a derivative together. Overload the arithmetic operators of Python so that ordinary code, written with no thought of derivatives, computes them when handed a dual argument – next frame, thirty lines. This is what autograd does to numpy.

Code: a Dual class (book Section 4.14.3)

```
class Dual:
    """A dual number a + b*eps with eps**2 = 0: a value and its derivative."""
    def __init__(self, val, dot=0.0):
        self.val, self.dot = val, dot
    def _lift(o):
        return o if isinstance(o, Dual) else Dual(o)
    def __add__(self, o):
        o = Dual._lift(o); return Dual(self.val + o.val, self.dot + o.dot)
    __radd__ = __add__
    def __sub__(self, o):
        o = Dual._lift(o); return Dual(self.val - o.val, self.dot - o.dot)
    def __mul__(self, o): # the product rule, Eq. (4.62)
        o = Dual._lift(o)
        return Dual(self.val * o.val, self.val * o.dot + self.dot * o.val)
    __rmul__ = __mul__

def sin(d): return Dual(math.sin(d.val), math.cos(d.val) * d.dot) # Eq. (4.68)
def log(d): return Dual(math.log(d.val), d.dot / d.val)

def f(x1, x2): # Eq. (4.61), ordinary code
    return log(x1) + x1 * x2 - sin(x2)

print("df/dx1 =", f(Dual(2.0, 1.0), Dual(5.0, 0.0)).dot) # 1/x1 + x2 = 5.5
print("df/dx2 =", f(Dual(2.0, 0.0), Dual(5.0, 1.0)).dot) # x1 - cos(x2)
```

Reverse mode: adjoints (Section 4.14.4)

Fix the *output* instead. For scalar y define the **adjoint** of every node, $\bar{v}_i = \partial y / \partial v_i$, Eq. (4.70). Since y depends on v_j only through the nodes that consume it,

$$\bar{v}_j = \sum_{i \succ j} \bar{v}_i \frac{\partial \varphi_i}{\partial v_j}, \quad j = \ell - 1, \dots, 1 - n, \quad \text{Eq. (4.71),}$$

started from $\bar{v}_\ell = 1$: the same local derivatives, summed over *successors*, running *backwards*. At (2, 5), Eq. (4.72):

$$\begin{aligned} \bar{v}_5 &= 1, & \bar{v}_4 &= 1, & \bar{v}_3 &= -1, & \bar{v}_1 &= \bar{v}_2 = 1, \\ \bar{v}_0 &= \bar{v}_3 \cos v_0 + \bar{v}_2 v_{-1} = 2 - \cos 5 = 1.716, & \bar{v}_{-1} &= \bar{v}_2 v_0 + \bar{v}_1 / v_{-1} = 5.5. \end{aligned}$$

Both partial derivatives from *one* sweep. The two terms in $\bar{v}_0$ are the two paths from v_0 to y in the graph: the chain rule sums over paths, and the reverse sweep visits each edge once.

The price: the tape

The local derivatives need the *values* v_j ($\cos v_0$, v_{-1}), in reverse order of production. The forward evaluation must be recorded; memory $\propto$ length of the trace. This is why training a deep network is memory-bound: every activation of every layer is stored until the backward pass has consumed it.

The cheap gradient principle (Section 4.14.4)

For a vector output seeded with a covector $\bar{y}$ the reverse sweep returns $\bar{x} = J^T \bar{y}$, a *vector–Jacobian product*, Eq. (4.73) – the chain rule of Eq. (1.51) with the products taken in the cheapest order.

Proposition 4.2 (Baur–Strassen, Linnainmaa)

Let $f : \mathbb{R}^p \rightarrow \mathbb{R}$ be evaluated by a trace of ℓ elementary operations with at most two arguments each. Then reverse-mode AD computes $f(x)$ and the complete gradient $\nabla f(x)$ in at most $c\ell$ operations, $c \leq 4$, **independent of p** .

Proof sketch. The forward evaluation costs ℓ and records the tape. In Eq. (4.71) each edge $j \rightarrow i$ contributes one term – one multiplication, one addition, a local derivative that is often free. At most two incoming edges per node, so at most 2ℓ edges. Nowhere did p enter: it decides how many adjoints are *read off* at the end, not how many operations are performed. $\square$ In practice $c \approx 2\text{--}3$: **a gradient costs two or three function evaluations**, against $p + 1$ for finite differences and p sweeps for forward mode. For $p = 10^7$ this is the difference between a fraction of a second and a week – the entire difference between what can and cannot be trained. Backpropagation is Eq. (4.71) specialised to the layered trace of a network.

Pause and think 2: which mode for which job?

Question

(a) Training a network with $p = 10^7$ parameters and a scalar loss. (b) A network with 3 inputs (x, y, t) whose output must satisfy a partial differential equation, so you need $\partial u / \partial x$, $\partial u / \partial t$ at many points. (c) The full $m \times n$ Jacobian of $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with $m \approx n$. Forward or reverse?

Pause and think 2: which mode for which job?

Question

(a) Training a network with $p = 10^7$ parameters and a scalar loss. (b) A network with 3 inputs (x, y, t) whose output must satisfy a partial differential equation, so you need $\partial u / \partial x$, $\partial u / \partial t$ at many points. (c) The full $m \times n$ Jacobian of $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with $m \approx n$. Forward or reverse?

Answer

Section 4.14.5 writes the Jacobian as the product $J = J_L \cdots J_1$, Eq. (4.74); forward multiplies from the right (cost $\propto n$ sweeps), reverse from the left ($\propto m$ sweeps). (a) $m = 1$, $n = 10^7$: reverse, by a factor 10^7 . (b) $n = 3$ inputs, many outputs: forward – three sweeps, and `jvp` gives each directional derivative directly (Chapter 9 uses exactly this). (c) Neither wins; the cheapest bracketing of Eq. (4.74) is NP-hard in general, use whichever and do not worry. Bonus: forward over reverse gives Hessian–vector products Hv , Eq. (4.75), for the price of a few gradients and without ever forming H – the curvature Newton’s method wanted, at a linear price.

JAX: grad, jit, vmap (Section 4.14.7)

```
import jax
jax.config.update("jax_enable_x64", True) # double precision throughout -- ALWAYS
import jax.numpy as jnp
from jax import grad, jit, vmap

def f(x):
    return jnp.sin(2 * jnp.pi * x + x**2)

df = grad(f) # reverse-mode gradient of a scalar function
print(df(1.0)) # 4.475424121402227, exact to all digits
print(grad(grad(f))(1.0)) # second derivative, by composing grad

xs = jnp.linspace(0.0, 1.0, 5)
print(vmap(df)(xs)) # the derivative at five points at once
fast_df = jit(df) # compiled on first call, then very fast
```

JAX combines the differentiation machinery with a compiler (XLA): the same code is differentiated, compiled, vectorised over a batch and run on a GPU. Forward and reverse mode are explicit as `jvp` and `vjp`, `jacfwd` and `jacrev`; `jax.hessian` is `jacfwd(jacrev(f))`.

Precision

JAX uses single precision by default. Without the first line, the gradient checks that follow agree only to seven digits, and one would wrongly conclude that AD is inexact.

The least-squares gradient without deriving it

```
Xj, yj = jnp.asarray(X), jnp.asarray(y) # the data of Section 4.4

def cost(theta, X, y): # Eq. (4.12), ordinary code
    return jnp.sum((X @ theta - y)**2) / len(y)

cost_grad = jit(grad(cost, argnums=0)) # d cost / d theta, compiled

theta = jnp.asarray(rng.normal(size=(2, 1)))
g_ad = cost_grad(theta, Xj, yj)
g_hand = (2.0 / n) * X.T @ (X @ np.asarray(theta) - y) # Eq. (4.13)
print("max |AD - hand| =", np.max(np.abs(np.asarray(g_ad) - g_hand))) # 5e-15

gamma = 0.1
@jit
def gd_step(theta): # Eq. (4.15) with the automatic gradient
    return theta - gamma * grad(cost)(theta, Xj, yj)
for k in range(1000):
    theta = gd_step(theta)
```

The two gradients agree to 5×10^{-15} – rounding – and the iteration returns the analytical (4.03012, 2.80580). **Do this check once, as a habit:** comparing an analytical gradient against the automatic one is the standard way of catching an error in a derivation. Project 1, part e), asks for exactly this comparison, for OLS and for Ridge – and last week's exercise 4d) did it.

Ridge, and a derivative nobody asked for

```
def ridge_cost(theta, X, y, lambda): # Eq. (4.16)
    return jnp.sum((X @ theta - y)**2) / len(y) + lambda * jnp.sum(theta**2)

lambda = 0.001
theta = jnp.asarray(rng.normal(size=(2, 1)))
step = jit(lambda th: th - gamma * grad(ridge_cost)(th, Xj, yj, lambda))
for k in range(1000):
    theta = step(theta)

theta_closed = np.linalg.inv(X.T @ X + n * lambda * np.eye(2)) @ X.T @ y # the n again
print("gradient descent:", np.asarray(theta).ravel())
print("closed form: ", theta_closed.ravel())

# the derivative with respect to lambda, argument number 3
dC_dlambda = grad(ridge_cost, argnums=3)(theta, Xj, yj, lambda)
print(dC_dlambda, "=", jnp.sum(theta**2)) # theta^T theta, by Eq. (4.16)
```

Changing the cost changes *one* function; the gradient follows. And grad differentiates with respect to *any* argument: $\partial C / \partial \lambda = \theta^T \theta$ falls out for free – the kind of quantity one wants when tuning hyperparameters by gradient methods rather than by grid search.

Hessians, one Newton step, and Hessian–vector products

```
from jax import hessian, jvp

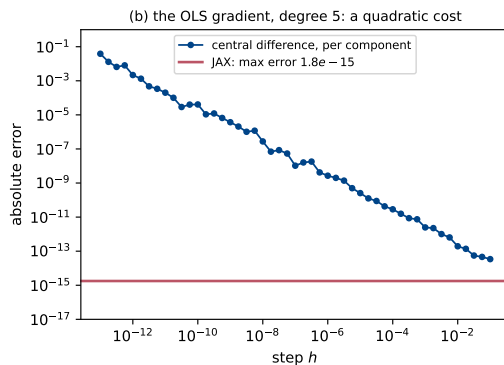
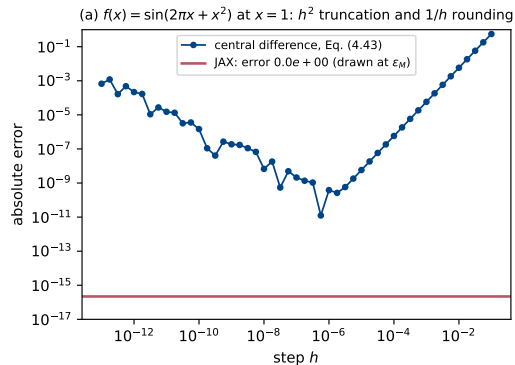
H = hessian(cost)(theta, Xj, yj).reshape(2, 2) # forward-over-reverse
print(H); print((2.0 / n) * X.T @ X) # Eq. (4.14), to rounding

theta0 = jnp.asarray(rng.normal(size=(2, 1))) # one Newton step, Eq. (4.8)
theta_newton = theta0 - jnp.linalg.solve(H, grad(cost)(theta0, Xj, yj))
print("one Newton step:", np.asarray(theta_newton).ravel()) # the exact minimum

def hvp(f, theta, v): # Eq. (4.75): H v without H
    return jvp(grad(f), (theta,), (v,))[1]
```

Everything this morning promised is now a few lines: the Hessian to rounding, the single exact Newton step of Eq. (4.9), and – for the problems where H cannot be stored – Hv for the price of a few gradient evaluations. Book Section 4.14.8 runs gradient descent, momentum, Adam and Newton on the non-convex Rosenbrock function with nothing derived by hand: 27 206, 2 591, 2 934 and 6 iterations. Replace the function by a network with a million parameters and only its definition changes.

Finite differences against automatic differentiation, measured



(a) $f(x) = \sin(2\pi x + x^2)$ at $x = 1$, book Section 4.14.9: the error falls as h^2 until $h \approx 10^{-6}$, then rises as $1/h$ – best 10^{-11} . JAX returns 4.475424121402227, every digit correct. **(b)** The degree-5 OLS gradient of last week's exercises, one central difference per component: the JAX gradient agrees with Eq. (4.13) to 1.8×10^{-15} for any h , because it involves no h . Whenever you use a finite-difference check (Eq. 4.77), h must sit at the bottom of the valley – and agreement better than 10^{-8} should not be expected.

Pause and think 3: the missing branch

Question

Panel (b) has no rising h^2 branch on the right: the finite difference of the OLS gradient is *best* at $h = 0.1$, with error 3×10^{-14} . Why – and does that make finite differences good enough for Project 1?

Pause and think 3: the missing branch

Question

Panel (b) has no rising h^2 branch on the right: the finite difference of the OLS gradient is *best* at $h = 0.1$, with error 3×10^{-14} . Why – and does that make finite differences good enough for Project 1?

Answer

The OLS cost is a quadratic, so $f''' \equiv 0$ and the truncation term of Eq. (4.59) vanishes identically: a central difference of a quadratic is exact for *any* h , and only rounding, $\propto 1/h$, remains. Ridge is quadratic too. So yes, for parts a)–f) finite differences would pass the check – and would still cost $2p$ cost evaluations per gradient against two or three for reverse mode, and would fail the moment the cost stops being quadratic (logistic regression in week 39: panel (a) is the general case). Use the finite difference as a *check* at $h \approx 10^{-5}$; use AD, or the analytical gradient, for the descent.

Why adapt the step size at all (Section 4.8)

The diagnosis of Section 4.5 once more, in the form that motivates what follows. A fixed γ is hard to get right: too large and Eq. (4.20) says it diverges, too small and Eq. (4.26) says it takes for ever. But the deeper problem is not the choice of a number:

No single number is right for all directions

The stability bound is set by $\lambda_{\max}$, the convergence rate by $\lambda_{\min}$. When they differ by orders of magnitude, the step that is barely safe along the steep direction is hopelessly small along the flat one. *Steep coordinates need small steps; flat coordinates could take large ones.*

Newton's method, Eq. (4.8), fixes this exactly by premultiplying with H^{-1} – and costs $\mathcal{O}(p^3)$. The three methods of this part are a compromise:

- ▶ approximate the curvature from quantities already available – the gradients themselves;
- ▶ restrict the approximation to a **diagonal** matrix, so that “inverting” it costs $\mathcal{O}(p)$;
- ▶ track the **second moment** of the gradient, g_j^2 , instead of the gradient alone.

The idea in one line: near a minimum $g_j \approx \lambda_j e_j$, so the typical size of g_j is a proxy for the curvature λ_j ; divide the step in coordinate j by $\sqrt{\langle g_j^2 \rangle}$ and you have imitated $H^{-1/2}$ without a second derivative.

AdaGrad, RMSProp and Adam differ only in how the average $\langle \cdot \rangle$ is taken.

The idea made precise: diagonal preconditioning (Section 4.8)

All three methods are instances of the **preconditioned** iteration

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \mathbf{D}_t^{-1} \nabla C(\boldsymbol{\theta}_t), \quad \mathbf{D}_t = \text{diag}(\sqrt{v_{t,1}}, \dots, \sqrt{v_{t,p}}) \quad \text{Eq. (4.41),}$$

Newton being $\mathbf{D} = \mathbf{H}$ and gradient descent $\mathbf{D} = \mathbf{I}$. For a quadratic cost the error recursion becomes $\mathbf{e}_{t+1} = (\mathbf{I} - \gamma \mathbf{D}^{-1} \mathbf{H}) \mathbf{e}_t$: what governs stability and speed is the spectrum of $\mathbf{D}^{-1} \mathbf{H}$, no longer that of $\mathbf{H}$.

Diagonal $\mathbf{H}$, $\mathbf{D}$ tracking its diagonal

$\mathbf{D}^{-1} \mathbf{H} = \mathbf{I}$: every direction contracts at the same rate, $\kappa \rightarrow 1$, the iteration count no longer depends on κ at all. This is the case of the $\kappa = 100$ bowl two frames ahead.

Off-diagonal $\mathbf{H}$

No diagonal $\mathbf{D}$ can do it. A valley at 45° with equally scaled coordinates has $\text{diag}(\mathbf{H}) \propto \mathbf{I}$: the preconditioner is a constant, and nothing is gained. Correlated columns $x, x^2, \dots, x^{10}$ are exactly this case.

Scale invariance – what the diagonal *does* buy

Multiply feature j by c : θ_j^* scales by $1/c$, the gradient g_j by c (chain rule), v_j by c^2 , $\sqrt{v_j}$ by c – the ratio $g_j/\sqrt{v_j}$ and hence the step in the rescaled units is *unchanged*. Plain gradient descent's step scales by c while the distance to the optimum scales by $1/c$, and its stability bound moves with c^2 .

Invariance to the *scaling* of features, not to their *correlation*.

AdaGrad: accumulate everything (Section 4.9)

Let $g_t = \nabla C(\theta_t)$ (possibly a minibatch gradient) and $r_0 = 0$. Accumulate the squared gradients coordinate by coordinate,

$$r_t = r_{t-1} + g_t \circ g_t, \quad \text{Eq. (4.42),}$$

and scale the step by the inverse square root, Eqs. (4.43)–(4.44):

$$\theta_{t+1,j} = \theta_{t,j} - \frac{\gamma}{\sqrt{\epsilon + r_{t,j}}} g_{t,j} \quad \text{Eq. (4.45).}$$

Each parameter has its own learning rate $\alpha_{t,j} = \gamma / \sqrt{\epsilon + r_{t,j}}$, which *never increases*.

What it buys

Parameters with large or volatile gradients take smaller steps, those with small or rare gradients relatively larger ones. A built-in, per-coordinate decaying schedule with no t_0, t_1 to tune. Designed for *sparse* features: a rare feature's $r_{t,j}$ stays small, so when it finally appears the parameter takes a large, useful step. In convex problems: a rate comparable to the best fixed γ chosen in hindsight.

What it costs

r_t grows without bound, so $\alpha_{t,j} \rightarrow 0$ – possibly long before the minimum. The sum has an *infinite memory*: it still remembers the huge gradients of the first steps, from a region the iterate left long ago. On a convex problem this is tolerable; in deep learning it is fatal – AdaGrad simply stops. Book Table 4.1: cost 28.05 at $\gamma = 0.01$ where everything else reaches 1.15.

AdaGrad's built-in schedule, and what forgetting it costs

AdaGrad along a direction of constant $|g|$

$r_t \approx t g^2$, so the effective learning rate is

$$\alpha_t = \frac{\gamma}{\sqrt{\epsilon + r_t}} \approx \frac{\gamma}{|g|\sqrt{t}} \quad \text{Eq. (4.46) :}$$

a $1/\sqrt{t}$ schedule, scaled by the gradient size so that every direction moves by about $\gamma/\sqrt{t}$. First step: $r_1 = g_1^2$, update $\gamma \text{sign}(g_1)$ – a distance γ in every coordinate. Cumulative displacement $\sum_t \gamma/\sqrt{t} \sim 2\gamma\sqrt{t}$: unbounded, ever slower.

On the Runge degree-6 fit (Tuesday, Case 1): RMSProp reaches an excess cost of 10^{-4} at best and never 10^{-6} , for every constant γ from 10^{-3} to 1; AdaGrad – the same idea *with* the $1/\sqrt{t}$ decay – converges to rounding. Forgetting removes the stall and the ability to settle in one stroke; RMSProp needs a decaying γ (Part 4) or a first moment (Adam) to converge.

RMSProp on $C = \frac{1}{2}\lambda\theta^2$ at constant γ

Near the minimum $v_t \approx g_t^2$ and

$$\theta_{t+1} \approx \theta_t - \gamma \text{sign}(g_t) \quad (g_t^2 \gg \epsilon) \quad \text{Eq. (4.50) :}$$

a step of *fixed* length γ . Convergence needs $\gamma\lambda/\sqrt{v_t + \epsilon} < 2$, but $v_t \rightarrow 0$ with θ_t , so the condition becomes $\gamma\lambda < 2\sqrt{\epsilon} = 2 \times 10^{-4}$: useless. Result: a limit cycle of amplitude $\sim \gamma$, excess cost of order $\lambda\gamma^2$, whatever λ and ρ .

RMSProp: forget the distant past (Section 4.10)

Replace the sum of Eq. (4.42) by an exponentially decaying average,

$$\mathbf{v}_t = \rho \mathbf{v}_{t-1} + (1 - \rho)(\nabla C(\boldsymbol{\theta}_t))^2, \quad \text{Eq. (4.47),} \quad \rho = 0.9 \text{ or } 0.99,$$

$$\boxed{\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\gamma}{\sqrt{\mathbf{v}_t + \epsilon}} \nabla C(\boldsymbol{\theta}_t)} \quad \text{Eq. (4.48),}$$

squares and divisions element-wise. Unrolling, Eq. (4.49),

$$\mathbf{v}_t = (1 - \rho) \sum_{j=0}^t \rho^{t-j} \mathbf{g}_j^2 :$$

a weighted *average* (weights summing to $1 - \rho^{t+1} \approx 1$), not an unbounded sum. Gradients older than about $1/(1 - \rho)$ steps – 10 for $\rho = 0.9$, 100 for $\rho = 0.99$ – are forgotten.

Two consequences

$\mathbf{v}_t$ now estimates the *recent* mean square gradient, so it tracks the local curvature as the iterate moves; and since it does not grow, the effective learning rate does not decay to zero. AdaGrad's infinite memory is gone.

Hinton, lecture notes, 2012, never formally published. Note the resemblance to momentum, Eq. (4.28): both are exponential moving averages – momentum of the gradient, RMSProp of its square. What if one does both?

Adam: both moments, and a correction (Section 4.11)

Adam – *adaptive moment estimation*, Kingma and Ba 2014 – keeps running averages of the first and the second moment of the gradient:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla C(\boldsymbol{\theta}_t) \quad (\text{momentum term}) \quad \text{Eq. (4.51),}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla C(\boldsymbol{\theta}_t))^2 \quad (\text{RMS term}) \quad \text{Eq. (4.52),}$$

$$\beta_1 = 0.9, \beta_2 = 0.999, \mathbf{m}_0 = \mathbf{v}_0 = 0.$$

Why both? They repair different defects

Momentum smooths the gradient: it accelerates along the consistent direction and damps the oscillation across the valley (Eq. 4.30). RMSProp rescales per coordinate: features of different scale, sparse gradients. Together: the *direction* of the step from an averaged gradient, its *length in each coordinate* from the recent curvature estimate.

The update, Eq. (4.55):

$$\boxed{\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\gamma}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t} \quad \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad \text{Eq. (4.54),}$$

with $\epsilon = 10^{-8}$. The hats are the **bias correction** – next frame, because it looks arbitrary and is not.

Bias correction, derived (Section 4.11)

Why divide by $1 - \beta_2^t$? Suppose the gradient is stationary with true second moment $\mathbb{E}[g^2]$. Unroll Eq. (4.52) as we unrolled RMSProp and take the expectation:

$$\mathbb{E}[v_t] = (1 - \beta_2) \sum_{j=1}^t \beta_2^{t-j} \mathbb{E}[g^2] = \mathbb{E}[g^2] (1 - \beta_2^t), \quad \text{Eq. (4.53),}$$

by the finite geometric sum. The raw average is too small by *exactly* the factor $1 - \beta_2^t$ – because it started from zero – and the same holds for m_t with β_1 .

How bad, at the start

With $\beta_2 = 0.999$ the factor is 10^{-3} at $t = 1$: v_1 underestimates the second moment by three orders of magnitude, $\sqrt{v_1}$ by a factor 30, and the step $\gamma m / \sqrt{v}$ would be thirty times too long. Dividing by the known factor removes the bias exactly; for large t , $1 - \beta_i^t \rightarrow 1$ and the correction disappears.

Defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\gamma = 10^{-3}$ are used almost universally – a large part of Adam's popularity.

Adam against its predecessors

AdaGrad: per-coordinate scaling, no momentum, never forgets. RMSProp: forgets, but no momentum and no bias correction. Adam = RMSProp + momentum + bias correction: acceleration and smoothing from $\hat{m}$, per-dimension step length from $\hat{v}$, sound estimates from the first iteration.

The size of an Adam step (Section 4.11)

The first step

At $t = 1$: $\hat{m}_1 = g_1$, $\hat{v}_1 = g_1^2$, so $\Delta\theta_1 = -\gamma \text{sign}(g_1)$ – a distance γ in every coordinate, as for AdaGrad.
Without the correction: $\gamma(1 - \beta_1)/\sqrt{1 - \beta_2} = 3.2\gamma$.
RMSProp, which has no correction, starts with $v_1 = (1 - \rho)g_1^2$ and a step of $\gamma/\sqrt{1 - \rho} = 10\gamma$.

Every step: a trust region

$\hat{m}_t$ is a weighted average of $g_1, \dots, g_t$, $\hat{v}_t$ of their squares, so $|\hat{m}_{t,j}|/\sqrt{\hat{v}_{t,j}} \lesssim 1$ and

$$|\Delta\theta_{t,j}| \lesssim \gamma \quad \text{Eq. (4.56).}$$

γ is the largest distance a parameter moves per iteration, in its own units. No γ can violate Eq. (4.20); the default 10^{-3} is a length, and transfers between problems.

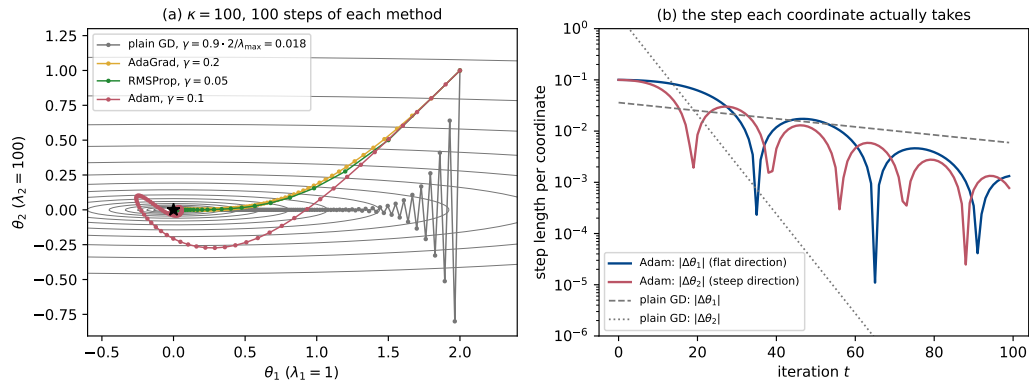
Why Adam settles where RMSProp cannot

Near a minimum the gradient alternates in sign; its average $\hat{m}_t$ is much smaller than its typical size $\sqrt{\hat{v}_t}$, so the ratio in Eq. (4.55) $\rightarrow 0$ at a *constant* γ . The fixed-length step of Eq. (4.50) has been averaged away. On the degree-6 Runge fit Adam converges for every γ in $[10^{-3}, 1]$, fastest at 0.1.

The caveat

The convergence proof in the 2014 paper is wrong; Reddi, Kale and Kumar (2018) built convex problems on which Adam does not converge, because $\gamma/\sqrt{\hat{v}_t}$ can grow when a large gradient is forgotten. AMSGrad keeps the running maximum of $\hat{v}_t$. Rare in practice; Adam stays the default.

What “adaptive” looks like on a bowl with $\kappa = 100$



$C = \frac{1}{2}(\theta_1^2 + 100\theta_2^2)$ from $(2, 1)$. **(a)** Plain gradient descent at $0.9\gamma_{\max} = 0.018$ zigzags down the steep θ_2 and crawls along the flat θ_1 (546 iterations to 10^{-4}). The adaptive methods normalise each coordinate by its own gradient history and set off *diagonally*, both coordinates moving at about γ per step: AdaGrad 297, Adam 151, RMSProp 52 iterations; momentum at $\gamma = 0.018$: 162. **(b)** The step each coordinate actually takes. Plain descent: $|\Delta\theta_2|$ falls as $|1 - \gamma\lambda_2|^t = 0.8^t$, $|\Delta\theta_1|$ as 0.982^t – the flat direction is starved. Adam gives both coordinates comparable steps until they are near the minimum, where $\hat{m}$ averages the sign changes away.

Pause and think 4: rotate the bowl

Question

Rotate the same bowl by 45° , so that its Hessian $Q \text{diag}(1, 100) Q^T$ has large off-diagonal entries. κ is unchanged. Which of the five methods change their iteration count?

Pause and think 4: rotate the bowl

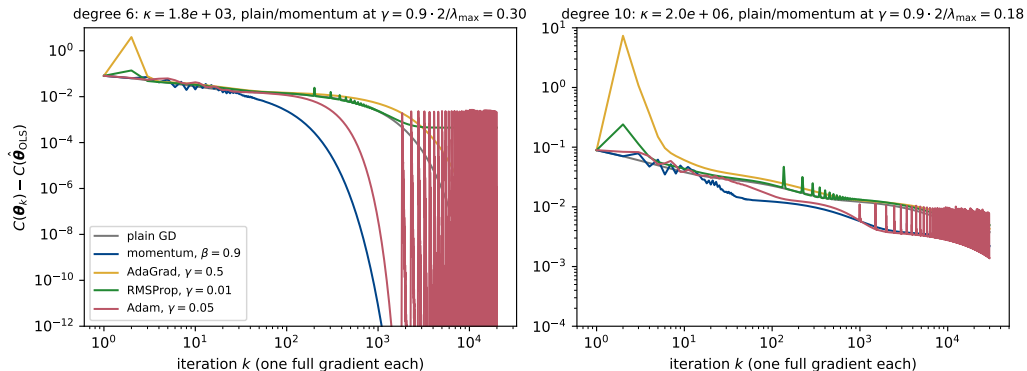
Question

Rotate the same bowl by 45° , so that its Hessian $Q \text{diag}(1, 100) Q^T$ has large off-diagonal entries. κ is unchanged. Which of the five methods change their iteration count?

Answer

Plain gradient descent and momentum: *unchanged*, 546 and 162 – Eq. (4.19) is a statement about eigenvalues, and rotations do not touch them. The adaptive methods: AdaGrad and RMSProp no longer reach 10^{-4} in 5000 iterations, Adam needs 635 instead of 151. They rescale *coordinates*, so they can only imitate the part of $H^{-1/2}$ that is aligned with the axes; a valley running diagonally has a Hessian whose curvature is in the off-diagonal entries, and no diagonal rescaling sees it. This is the residual gap between Adam and a second-order method, and it is why *decorrelating* inputs helps even when their scales already match. Keep it in mind for polynomial features: the columns $x, x^2, \dots, x^{10}$ are strongly correlated, so their valley is not axis-aligned. Next frame.

Last week's unsolved problem: Runge at degree 10, $\kappa = 2 \times 10^6$



Excess cost $C(\theta_k) - C(\hat{\theta}_{OLS})$, full gradients, $\theta_0 = 0$. **(a)** Degree 6, $\kappa = 1849$: iterations to 10^{-8} – plain 7346, AdaGrad 12 156, Adam ($\gamma = 0.05$) 1062, momentum 670; RMSProp at a constant γ never settles (its step is $\approx \gamma \text{sign}(g)$ near the minimum – it needs a decaying γ , which AdaGrad has built in and Adam gets from $\hat{m} \rightarrow 0$). **(b)** Degree 10, $\kappa = 2 \times 10^6$: Adam reaches 10^{-2} in 462 iterations against 2793 for plain descent – and then *everything* stalls between 10^{-3} and 5×10^{-3} for 30 000 iterations. The flat directions of a correlated polynomial basis are not axis-aligned (previous frame), and no first-order method today gets there. One Newton step with the JAX Hessian does (week 37, Case 2); so does Ridge with $\lambda = 10^{-3}$ ($\kappa \rightarrow 5000$).

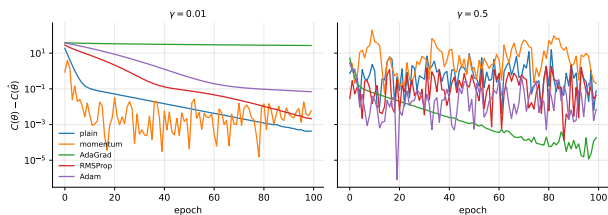
Code: one function, five optimisers

```
def optimiser_step(method, theta, g, state, t, gamma, beta=0.9, rho=0.99,
                  beta1=0.9, beta2=0.999, eps=1e-8):
    """One update of theta from the gradient g at step t = 1, 2, ...; state carries the running
        quantities."""
    if method == "plain":
        return theta - gamma * g, state
    if method == "momentum":
        state["v"] = v = beta * state.get("v", 0.0) + gamma * g # Eq. (4.28)
        return theta - v, state
    if method == "adagrad":
        state["r"] = r = state.get("r", 0.0) + g * g # Eq. (4.42)
        return theta - gamma * g / (np.sqrt(r) + eps), state # Eq. (4.45)
    if method == "rmsprop":
        state["r"] = r = rho * state.get("r", 0.0) + (1.0 - rho) * g * g # Eq. (4.47)
        return theta - gamma * g / (np.sqrt(r) + eps), state # Eq. (4.48)
    if method == "adam":
        state["m"] = m = beta1 * state.get("m", 0.0) + (1.0 - beta1) * g # Eq. (4.51)
        state["r"] = r = beta2 * state.get("r", 0.0) + (1.0 - beta2) * g * g # Eq. (4.52)
        m_hat, r_hat = m / (1.0 - beta1**t), r / (1.0 - beta2**t) # Eq. (4.54)
        return theta - gamma * m_hat / (np.sqrt(r_hat) + eps), state # Eq. (4.55)
```

Same three-function structure as last week: cost, gradient (by hand or by `jax.grad`), one step function. Twenty lines hold Sections 4.6–4.11; book Section 4.12 has them in the epoch/minibatch form of Part 4. Only Adam needs to know the step number t .

A learning rate is not comparable across optimisers (book Figure 4.7)

Book Section 4.12: the five optimisers on the $p = 2$ least-squares problem of Section 4.4, 100 epochs, batches of 10; excess cost against epoch.



	$\gamma = 0.01$	0.1	0.5
Plain SGD	1.1499	1.1532	1.2274
Momentum	1.1553	1.5674	1.3490
AdaGrad	28.049	1.2776	1.1497
RMSProp	1.1516	1.1512	1.1904
Adam	1.2185	1.1514	1.1565

Minimum 1.14947. **No row is best at the same γ as any other.** Plain descent multiplies γ by the raw gradient, so Eq. (4.20) applies directly; the adaptive methods first divide by $\sqrt{v_j}$, which renormalises the gradient to order one, so their γ is a *step length in parameter space*. A comparison at one fixed γ says which method happens to suit that number. And on this benign problem ($\kappa = 12$) the sophisticated methods are not better: there is nothing for them to repair.

Pause and think 5: what is γ , then?

Question

Adam with $\gamma = 0.1$ took its first step of length about 0.1 in *every* coordinate on the $\kappa = 100$ bowl, whatever the gradient was. Why – and what does that make γ for an adaptive method?

Pause and think 5: what is γ , then?

Question

Adam with $\gamma = 0.1$ took its first step of length about 0.1 in *every* coordinate on the $\kappa = 100$ bowl, whatever the gradient was. Why – and what does that make γ for an adaptive method?

Answer

At $t = 1$, $\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$ after bias correction, so the update is $\gamma g_1 / (|g_1| + \epsilon) = \gamma \text{sign}(g_1)$: a step of length γ per coordinate, independent of the gradient's magnitude – and by the trust-region bound Eq. (4.56) no later step is longer. So γ is the *largest distance a parameter moves per iteration*, a length in parameter space, which is why the defaults (10^{-3} for Adam) are so much smaller than a gradient descent learning rate, why they transfer between problems as gradient descent's γ never does, and why these methods are invariant to rescaling a feature (the gradient scales by c , the root of the second moment by c , the ratio not at all). Book Table 4.1 is this fact in numbers: the same γ is a multiplier for plain descent and a length for Adam, so no row is best at the same γ as any other.

None of these can compete with Newton's method (Section 4.13)

Every method of this part is a step back towards Eq. (4.8),

$$\theta_{k+1} = \theta_k - H^{-1} \nabla C(\theta_k),$$

recovering a little more curvature for a little more bookkeeping:

plain GD	$H^{-1} \rightarrow \gamma I$
momentum	+ memory of the gradient
AdaGrad/RMSProp	$H^{-1} \rightarrow \gamma \text{diag}(v)^{-1/2}$
Adam	both, bias-corrected
Newton	H^{-1} , $\mathcal{O}(p^3)$

The quality of a method can fairly be judged by how much of H it imitates for $\mathcal{O}(p)$ work per step.

Book Section 4.14.8: Rosenbrock, nothing derived by hand

$f = (1 - x)^2 + 100(y - x^2)^2$, $\kappa \approx 2500$ at the minimum; gradient and Hessian from JAX. Iterations to $f < 10^{-10}$: plain 27 206, momentum 2 591, Adam 2 934, Newton 6.

Machine-learning connection

Adam is an excellent default for getting a model to train at all. Several studies find that well-tuned SGD with momentum generalises *better* than Adam, particularly when $p > n$; why is not settled. Try both before reporting a result.

Stochastic gradient descent: attacking the cost (Section 4.7)

Almost every cost in machine learning is a **sum over data points**, Eq. (4.32):

$$C(\boldsymbol{\theta}) = \sum_{i=1}^n c_i(\mathbf{x}_i, \boldsymbol{\theta}), \quad \nabla C = \sum_{i=1}^n \nabla c_i(\mathbf{x}_i, \boldsymbol{\theta}).$$

One full gradient costs a pass over the whole data set – for one update, most of whose precision is wasted, since we step again immediately.

Minibatches

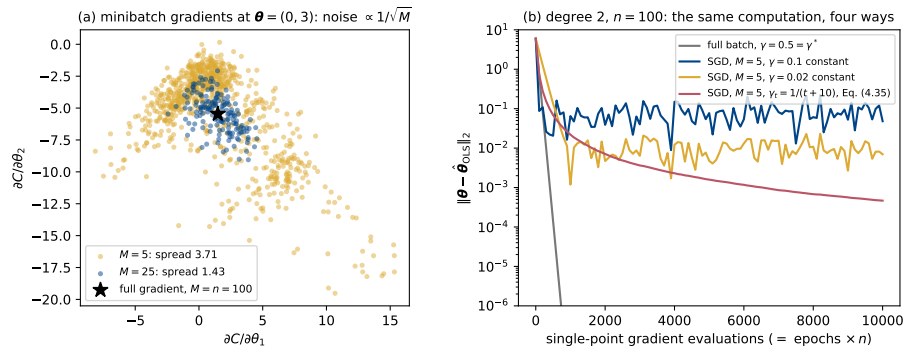
Split the n points into n/M *minibatches* B_k of size M , and replace the sum by the sum over one of them, Eq. (4.33),

$$\nabla C \longrightarrow \sum_{i \in B_k} \nabla c_i(\mathbf{x}_i, \boldsymbol{\theta}), \quad \boldsymbol{\theta}_{j+1} = \boldsymbol{\theta}_j - \gamma_j \sum_{i \in B_k} \nabla c_i(\mathbf{x}_i, \boldsymbol{\theta}_j) \quad \text{Eq. (4.34),}$$

k drawn at random. One pass through all minibatches is an **epoch**. $M = n$: gradient descent. $M = 1$: stochastic gradient descent in the strict sense. Everybody says SGD and means minibatches; so shall we.

M is a hyperparameter, but seldom cross-validated: set by memory, or to a power of two (32, 64, 128) because vectorised hardware is fastest there – one gradient on 100 points costs far less than 100 single-point ones.

What the minibatch gradient is: an unbiased, noisy estimate



(a) Last

week's degree-2 data, $n = 100$, at the point $\theta = (0, 3)$: the cloud of minibatch gradients (scaled by n/M) for $M = 5$ and $M = 25$, and the full gradient. The cloud is centred on the full gradient – the estimate is *unbiased* – and its spread falls as $1/\sqrt{M}$ (Section 2.5, the central limit theorem): 3.7 against 1.4. Quadrupling M halves the noise at four times the cost per step, which is why huge batches are not automatically better. (b) Distance to $\hat{\theta}_{OLS}$ against the number of single-point gradient evaluations: full-batch descent at $\gamma^* = 0.5$ (11 iterations = 1100 evaluations, then rounding); SGD with $M = 5$ at constant γ is ahead for the first few hundred evaluations and then *rattles* at a distance set by γ and the noise (5×10^{-2} at $\gamma = 0.1$, 10^{-2} at 0.02); the schedule of Eq. (4.40) keeps descending, 5×10^{-4} after 100 epochs.

Three ways to read the noise

1. Cost. Per iteration, gradient descent wins; per unit of computation, SGD does

Full-batch descent converges geometrically at the rate of Eq. (4.26). SGD with a decaying γ converges only as $\mathcal{O}(1/k)$ (strongly convex) or $\mathcal{O}(1/\sqrt{k})$ (in general) – the noise does not vanish. But each SGD step costs M/n of a full one: for $n = 10^6$ and $M = 100$, ten thousand updates for the price of one. And SGD needs only the current minibatch in memory – the reason training on data larger than memory is possible at all, and the reason every framework is built around minibatches.

2. Exploration. Noise jostles the iterate out of shallow basins

A deterministic iterate stops at the first stationary point it reaches (Part 1, panel (a)). A stochastic one is kicked around, with a kick size $\propto \gamma/\sqrt{M}$; it can leave a narrow, shallow basin and settle in a wide one. Wide minima are also the ones whose cost does not change much when the data change – an empirical link between SGD noise and generalisation that is much discussed and not settled.

3. A random walk in a bowl. The iterate does not converge; it equilibrates

With constant γ , the iterate near the minimum is a random walk pulled back by the gradient: it settles into a cloud of radius $\propto \sqrt{\gamma} \sigma_g / \sqrt{M}$ around $\hat{\theta}$, panel (b). To shrink the cloud: decrease γ , or increase M – or, since nothing else is needed at the end, do both by *decaying* γ .

Schedules and stopping (Section 4.7.1)

Let $t = e \cdot m + i$ count updates (e the epoch, $m = n/M$ minibatches). A common schedule, Eq. (4.40): $\gamma_t = t_0/(t + t_1)$, from t_0/t_1 down to zero. **Robbins–Monro**: convergence needs $\sum_t \gamma_t = \infty$ (the iterate can still get anywhere) and $\sum_t \gamma_t^2 < \infty$ (finite accumulated noise); $1/t$ satisfies both, a constant neither.

```
def make_batches(n, batch_size, rng):
    idx = rng.permutation(n)
    return [idx[i:i + batch_size] for i in range(0,
        n, batch_size)]

def step_length(t, t0, t1): # Eq. (4.40)
    return t0 / (t + t1)

t = 0
for epoch in range(n_epochs):
    for batch in make_batches(n, M, rng):
        t += 1
        g = gradient(theta, X[batch], y[batch]) # Eq
            . (4.33)
        theta, state = optimiser_step(method, theta,
            g, state, t, step_length(t, t0, t1))
```

When do we stop?

The full gradient every few epochs, stop when small? That finds a stationary point, not a good one – record the cost there, continue, keep the best. The run is random by design: repeating it and keeping the best of several is legitimate and common. **The**

machine-learning criterion is not about the gradient at all: monitor the cost on a held-out validation set and stop when it starts to *rise* while the training cost still falls. **Early stopping** is a regularisation method as much as a termination rule: halting before the variance term of Eq. (2.52) has grown trades a little bias for a lot of variance, in the spirit of Ridge (Section 3.8).

`make_batches` reshuffles every epoch, so no minibatch is the same twice; the book's `randint` variant draws with replacement – also fine.

Pause and think 6: the same epochs, twice the noise

Question

You run SGD with $M = 5$ at $\gamma = 0.1$ and see the iterate rattle at 5×10^{-2} from the minimum. Name three changes that each shrink the rattle, and say what each costs.

Pause and think 6: the same epochs, twice the noise

Question

You run SGD with $M = 5$ at $\gamma = 0.1$ and see the iterate rattle at 5×10^{-2} from the minimum. Name three changes that each shrink the rattle, and say what each costs.

Answer

(1) Smaller γ : the cloud radius scales like $\sqrt{\gamma}$ (panel (b): 0.02 gives 10^{-2}) – costs speed early on, Eq. (4.26) with a smaller γ . (2) Larger M : noise $\propto 1/\sqrt{M}$ – costs M per step, so four times the work for half the radius, and fewer updates per epoch. (3) A decaying γ_t : both at once, at no cost per step, but with two new numbers t_0, t_1 to choose – and if they decay too fast (Tuesday, Case 2: the same (t_0, t_1) on the degree-5 data) the iterate freezes before it arrives, because $\sum \gamma_t$ is finite in practice. Bonus: Adam *with* minibatches is the default of every framework – its $\hat{m}$ averages the minibatch noise as well as the sign changes. And none of the three helps if the goal is a good *test* error rather than a precise $\hat{\theta}$: then early stopping is the criterion.

Summary of week 38

- ▶ **Gradient descent and momentum:** $\gamma < 2/\lambda_{\max}$ decides convergence, κ its length; momentum trades κ for $\sqrt{\kappa}$ (Eqs. 4.20, 4.26, 4.28–4.31).
- ▶ **Automatic differentiation** propagates exact derivative rules through the evaluation trace: forward mode (Eqs. 4.64–4.69), reverse mode (Eqs. 4.70–4.73), the cheap gradient principle – a gradient for two or three function evaluations, whatever p . `jax.grad` of your cost *is* your gradient; check it once against Eq. (4.13).
- ▶ **Adaptive methods** imitate $H^{-1/2}$ with a diagonal built from the second moment of the gradient: AdaGrad sums and never forgets (Eq. 4.45), RMSProp averages with decay (Eq. 4.48), Adam adds momentum and bias correction (Eqs. 4.51–4.55). Their γ is a step length; they see only axis-aligned curvature; on a benign problem they buy nothing.
- ▶ **Stochastic gradient descent** replaces the sum over n by a sum over a minibatch (Eqs. 4.32–4.34): unbiased, noise $\propto 1/\sqrt{M}$, far ahead per unit of computation; the iterate equilibrates rather than converges unless γ decays (Eq. 4.40); stop on validation error.

Tomorrow, the deadline, and next week

Tuesday: Case 1 = AdaGrad, RMSProp and Adam in your own code on the Runge problem (Project 1, part f); Case 2 = minibatches, epochs and schedules (part h). Exercises due **Friday September 18 at midnight**. Week 39: Lasso by gradient methods (part g) and logistic regression (Chapter 5).